

Third International Conference on Computing and Network Communications (CoCoNet'19)

Learning-Based Hole Detection in 3D Point Cloud Towards Hole Filling

Ramesh Ashok Tabib, Yashaswini V. Jadhav, Swathi Tegginkeri, Kiran Gani, Chaitra Desai, Ujwala Patil, Uma Mudenagudi

KLE Technological University, Vidyanagar, Hubli-580031, India.

Abstract

In this paper, we propose a learning-based approach for automatic detection of hole boundary points in a 3D point cloud. Point cloud is an important geometric data structure used in 3D modelling. Data obtained from automatic acquisition techniques often result in geometric deficiencies such as holes in the 3D point cloud. For successful hole-filling to achieve better surface reconstruction, accurate detection of hole boundary points in the point cloud is necessary. Most of the existing methods use threshold values for different parameters which need to be set manually after analyzing the nature of point cloud. It becomes difficult to generalize the threshold values for a wide variety of point clouds. To overcome this limitation, we propose a deep learning framework to detect holes in the point cloud. The architecture directly consumes point cloud and considers the permutation invariance of points. Each point in the point cloud is classified as a hole boundary point or not. The detected hole boundary points are used for hole filling by fitting a surface and interpolating points on the surface.

© 2020 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the Third International Conference on Computing and Network Communications (CoCoNet'19).

Keywords: Point cloud; Hole detection; Meshless method; Hole filling; Learning-based approach.

1. Introduction

Hole detection is a significant step of point cloud processing towards hole filling and inpainting of 3D point cloud for better surface reconstruction. 3D data acquisition has gained popularity due to the low-cost and easy availability of Time of Flight and Microsoft Kinect cameras. However, the scanned 3D data coming from an automatic acquisition technique usually results in geometric deficiencies like some parts of the objects can be missing due to problems with accessibility/visibility, or due to the special physical properties of the scanned surface like transparency and reflectivity [1]. It is possible to scan the complete surface of an object using high-fidelity scanners, but holes may appear in the

E-mail address: ramesh_t1@kletech.ac.in

final integrated model due to occlusions and maneuverability of the scanner while scanning. The holes in 3D objects which do not correspond to any holes in the object may also occur due to factors such as occlusion, low reflectance coefficients of the surface, high grazing angles, missing pieces in the original object, limited number of range scans from different viewing directions and functionality of the laser range scanner, etc.

1.1. Hole detection

Creating high-quality point cloud representations of objects based on such incomplete information remains a challenge [2]. Automatic and semiautomatic tools can be used to overcome the difficulties involved in scanning real environments to enhance the quality of incomplete datasets [3]. Although methods for triangulation of point cloud have recently made some progress, they are still difficult and computationally expensive. Also, they may not give desired results, especially for complicated and/or noisy data. Lack of connectivity information in point cloud data reduces the complexity of implementing different tasks involved in geometric modelling. Therefore, it simplifies the definition and implementation compared to mesh-based representation.

Existing state-of-art methods for hole detection are majorly based on polygonal representation, parametric representation, volumetric representation and signed distance function. Some of the other methods include 2D image-based methods and context-based methods [4]. Most of the above methods use the distance-centroid algorithm to detect neighborhood points and there is a need to manually select threshold values for parameters like angle criterion, ellipsoid criterion, weighted average criterion, half-disc criterion and shape criterion but it is a tedious task. These methods use threshold values for different parameters which need to be set manually after analyzing the nature of point cloud. It becomes difficult to generalize the threshold values for a wide variety of point clouds as different point clouds may have different distribution and density of points.

For a successful hole-filling in the point cloud, accurate detection of hole boundary points is necessary. Some of the previous methods employed either special equipment or triangular meshes, usually associated with input entered manually by users for hole detection. Normally, in a 3D point set, approaches for computing the boundary of a surface either uses its convex hull or convex envelope of a set of 3D points [5], [6], [7]. Other approaches are based on the calculation of angle or distance between points and their neighbors [8], [9]. Apart from the angle criterion which is used in many of the papers, Authors in [10] additionally use weighted average criterion and ellipsoid criterion to detect the hole boundary points. To overcome the limitations of not sampling the point cloud, the calculation of neighbourhood is performed using the symmetric function. Finally, all the criteria are combined to decide the hole boundary point. Authors in [11] propose some other criteria such as half-disc criterion and shape criterion along with the angle criterion.

Authors in [12] analyze the 3D structure of objects appearing in images by investigating the combination of the 3D structure together with the features and details provided by the images. Therefore, it tries to relate the 3D information, which is obtained after 3D reconstruction, with the 2D views. Authors in [13] propose a method to extract and simplify the boundary of an elevation surface defined into a 3D volume. Authors in [14] extract exterior boundary points of the surface S and further classify them into exterior boundary points and inter points. Then looping is done for all points of S , if each of them is not an exterior boundary point, and one of its neighbors (in 4-connectivity) is empty, it is determined as a boundary point of a hole.

In some of the hole-boundary identification algorithms for 3D closed triangle mesh, the user can select the region of interest interactively by mouse dragging. The meshing algorithms may fill the regions of missing depths which prevents distinguishing real holes from virtual holes. Therefore, dependence on mesh-based techniques cannot guarantee accurate detection of all holes. Authors in [15] use triangulated mesh for analysing the existence of boundary edges which are used to trace the entire boundary. A ring of points around the boundary is used for an interpolation procedure that will eventually fill in the hole using radial basis function (RBF) interpolator. Authors in [16] use the volumetric representation of the surface, which is a regularly spaced 3D grid of values of a clamped signed distance function. The algorithm uses both diffusion volume and source volume. Later convolution is performed to interpolate missing points.

Author in [17], detects holes by first finding the neighborhood of each point and estimating the direction of the normal. Then boundary is built by detecting candidate boundary points and constructing boundary polygons from boundary points. After detecting the boundary points of the hole, an algebraic surface patch is fit to the neighborhood and sample auxiliary points. The method reproduces surfaces such as planes, cylinders and spheres. Authors in [18],

use the Distance-Centroid algorithm to detect the boundary points which uses K-Nearest neighbors and maximum squared distance. Statistical Outlier Removal (SoR) is used to filter out the points which are not belonging to the hole boundary. Region growing segmentation is used to cluster out the different holes and leaving future scope to be indexing the holes so that only the selected holes are filled. Authors in [1] propose a hole-filling method based on feature extraction and decomposition of neighborhood region of the hole for which accurate hole boundary points detection is necessary.

With the advancement in deep learning, researchers have proposed deep learning architectures for point clouds which find applications like object detection, part segmentation and semantic segmentation [20].

1.2. Hole filling

Hole filling is an important problem and finds applications in 3D shape representation and analysis, surface reconstruction or model rebuilding, 3D model completion, etc. The polygonal surface is required to be closed, intersections-free and without singular points. It is very important in engineering and manufacturing, where solid objects are required for numerical computations and production task to meet these requirements. To fulfil the topological demands, all pieces of the polygonal model should follow connectivity rules and errors like disconnected patches and redundant handles should be avoided. These errors introduce complexity to the model and lead to complication of parametrization and segmentation tasks.

Authors in [21] use triangulated mesh to analyse the existence of boundary edges. The occurrence of a hole implies the existence of a polygon defined by boundary edges. Thus, the boundary edges are used to trace the entire boundary. The hole is filled by a ring of points around the hole boundary using the interpolation technique. The points near the hole are then used to fit a surface using a Radial Basis Function (RBF) interpolator. Authors in [22] use the volumetric representation of the surface, which is a regularly spaced 3D grid of values of a clamped signed distance function. The algorithm uses two volumes, the diffusion volume and the source volume. The diffusion volume is where the computation takes place and source volume represents the observed surface. Later the algorithm performs convolutions to interpolate the missing points. Sparse representation of volume is used to enhance computation by avoiding the use of memory for undefined areas and computation of voxels is limited to a particular distance from a hole boundary in the original surface.

To overcome the above mentioned limitations,

- We propose to model a deep learning framework towards hole boundary detection.
 - We model point cloud data towards learning hole boundary points.
 - We train the architecture to learn detection of hole boundary points using the modelled data.
- We propose a geometrical method for hole filling.
 - We fill the holes using the detected hole boundary points by surface fitting and point interpolation.
- We demonstrate the results of proposed framework using synthetic and real dataset.

The remainder of this paper is structured as follows: In Section 2, we present a deep learning framework for hole boundary detection. In Section 3, we present a methodology for hole filling. In Section 4, we illustrate our results which are gathered in every stage and the paper is concluded in Section 5.

2. Learning-based framework for hole boundary detection

Point clouds are simple and easy for a network to learn compared to the complexities of meshes. Therefore, we use a deep learning architecture which directly takes unordered point cloud as input, unlike other networks which use point cloud data transformed into regular 3D voxel grids or images from different views. The network directly takes point cloud as input and outputs label (hole boundary point or not) for each point in the input. Rotation and translation of points should not affect the classification of points.

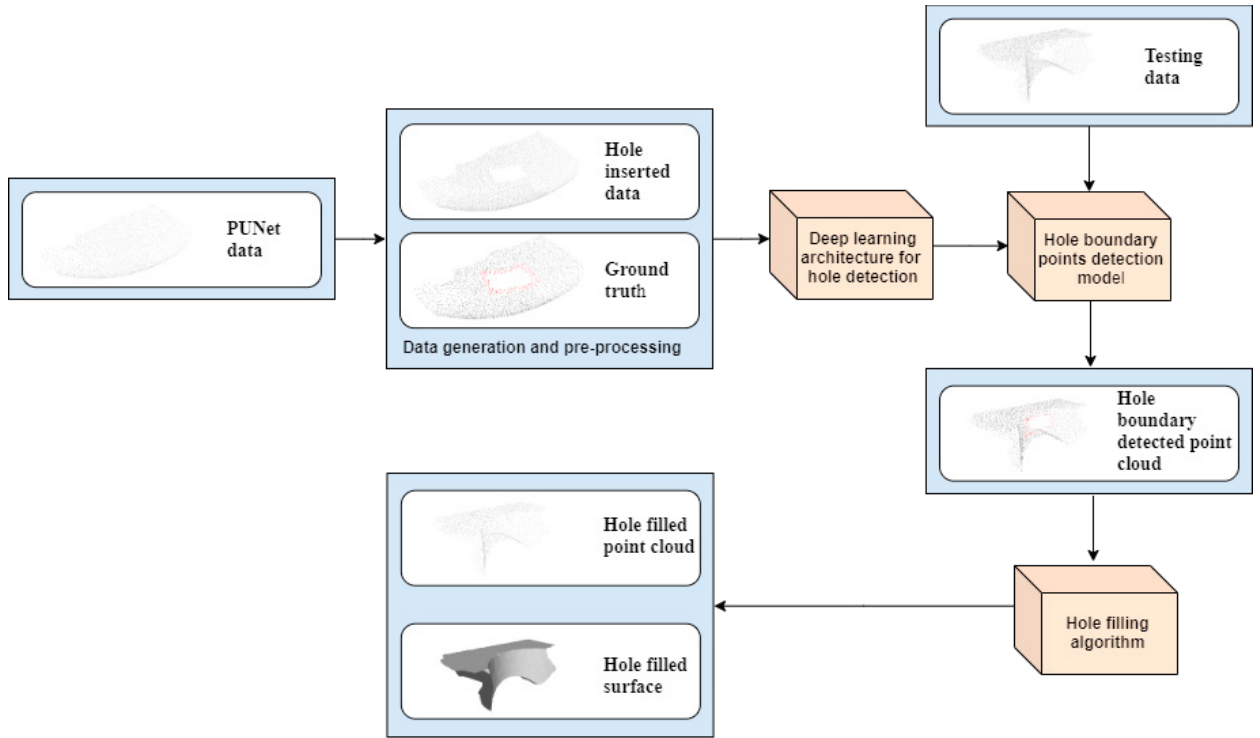


Fig. 1: Proposed framework for learning-based hole boundary detection

2.1. Data generation and pre-processing

Due to the unavailability of public benchmarks for hole detection in a 3D point cloud, we use PUNet dataset [19], a collection of sixty different models from Visionair repository. Then each point cloud is cropped into 100 patches of 4096 points each, which sums up to 4000 point clouds. To create hole dataset we introduce random holes into these point clouds. Out of these, we use 2400 for training, 800 for testing and 800 for validation.

2.2. DNN-Architecture for hole detection

A point cloud is represented as a set of 3D points $\{P_i \mid i = 1, \dots, n\}$, where each point P_i is a vector having (x, y, z) coordinates. Since the network takes n 3D points, it should be invariant to $n!$ permutations of the input data.

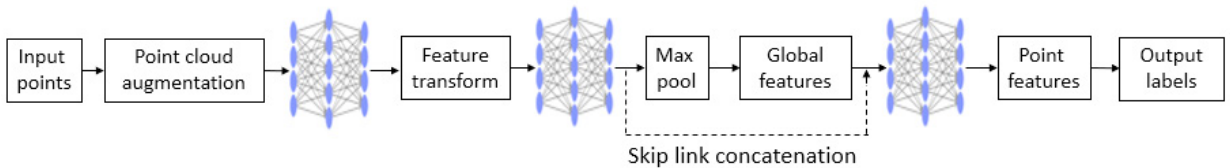


Fig. 2: Deep learning architecture

The main task of the network is to assign a label to each point in the input point cloud data. The label specifies if a point belongs to the hole boundary or not i.e., per-point classification. The network tries to learn a set of optimization functions that select hole boundary points of the 3D point cloud and encodes the reason for selecting them. Max pooling layer is used as the symmetric function to aggregate information from all the points. The symmetric function takes n vectors (each vector representing (x, y, z) coordinates of each point in the input point cloud) as input and

returns a new vector which is invariant to the order of input points. Once the global point cloud feature vector is computed, it is concatenated with each of the point features. Further new point features are extracted based on the combined point features which consider both the local and global information. Later input points and point features are aligned to a canonical space before feature extraction. Finally, the fully connected layers of the network combine the learned optimal values into the global descriptor to predict labels for each point.

Our framework classifies each point into hole boundary or not. Hole boundary points are given different RGB values compared to non-hole boundary points. For each point cloud, IoU (Intersection over Union) is computed between ground truth and prediction which is used as an evaluation metric on points.

3. Hole filling

The detected hole boundary points are further used in the hole filling algorithm. The hole boundary points are projected onto eigenspace using Principal Component Analysis (PCA). To get new axes, the points are multiplied with coefficients of PCA. Using only two axes, the points are projected to 2D eigenspace. Later, boundary fitting is performed on the projected hole boundary points and random points are generated inside the hole boundary limit based on density of points in the neighborhood. A fifth order surface is fit using hole boundary points and the generated points are interpolated on that surface. Finally, we project back the points to its original coordinate space and merge with the original point cloud to get the hole filled point cloud.

4. Results and Discussions

As mentioned above, hole detection has not been addressed before using a learning-based approach. However, many experiments have been conducted to detect and fill holes using non-learning geometrical approaches. In this section, we present implementation details and results to illustrate the algorithm to extract the hole boundary points. The architecture is trained for 200 epochs. The training mean loss is 0.2569 and training accuracy is 0.8861. Testing accuracy is 0.9037 and mIoU (mean Intersection over Union) obtained is 0.6281. Table 1 illustrates the results on the testing dataset of 10 point clouds showing each of its accuracy and IoU.

Table 1: Results on the testing dataset with accuracy and IoU

Point cloud ID	Accuracy	IoU
1	0.943848	0.726517
2	0.900879	0.623345
3	0.927246	0.672818
4	0.952148	0.746718
5	0.952637	0.746136
6	0.955566	0.712333
7	0.858887	0.615768
8	0.948242	0.681975
9	0.960938	0.861624
10	0.958008	0.749180

Figure 3a represents a point cloud with a hole. Figure 3b and Figure 3c represent mesh and the reconstructed surface of the point cloud respectively. This point cloud containing hole is fed to our architecture which results in detection of hole boundary points as shown in Figure 3d. The enlarged view of detected hole boundary is shown in Figure 3e. Further, the detected hole boundary points are used for hole filling, the output of which is shown in Figure 3f. The mesh representation and surface reconstruction of point cloud in Figure 3f is shown in Figure 3g and Figure 3h respectively. Similar results are illustrated with different point cloud in Figure 4. The next set of images in Figure 5 illustrate the detection results on real-time data [23].

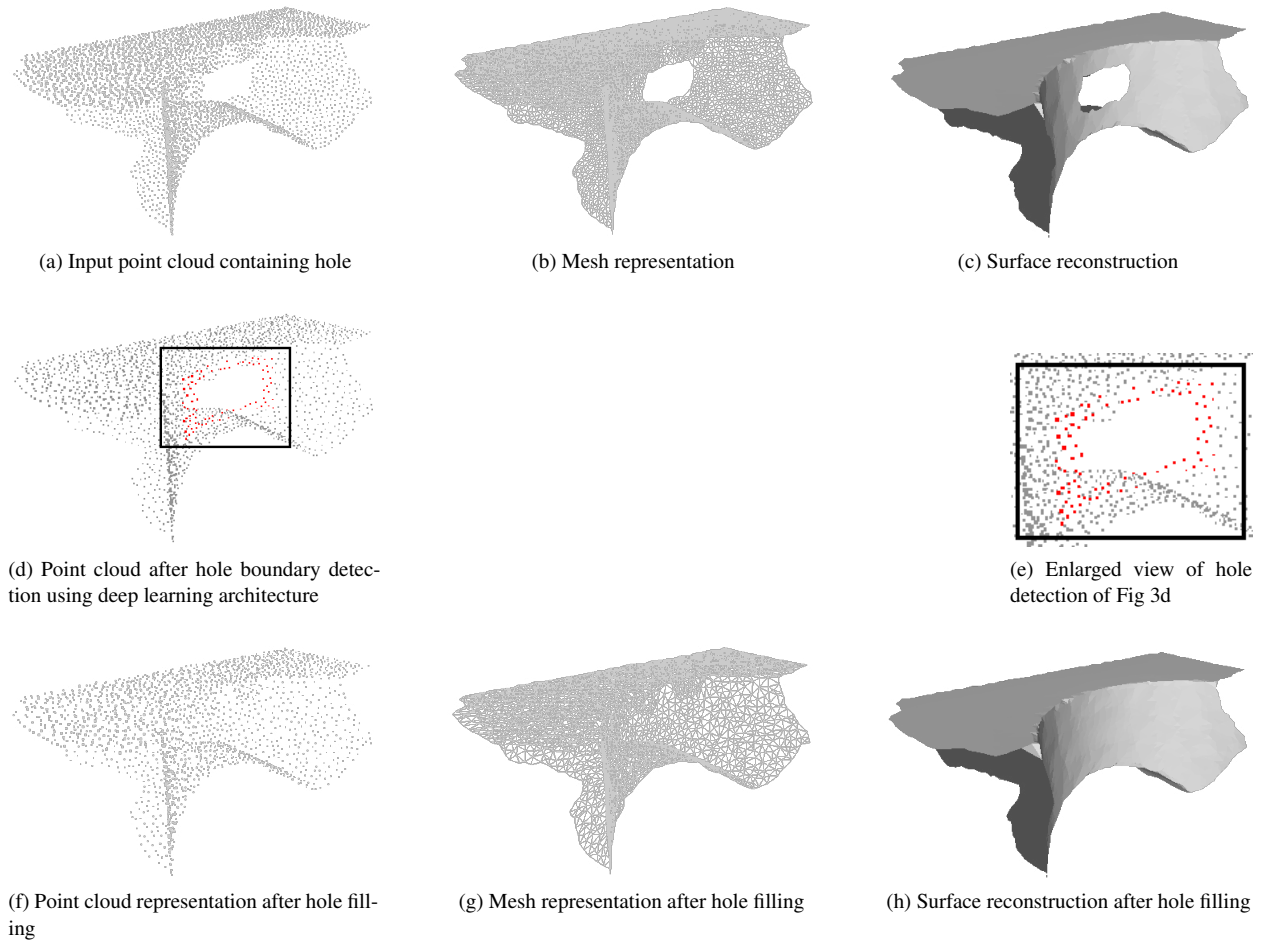


Fig. 3: Illustration of hole detection and filling in point cloud using our proposed framework

5. Conclusions

We have proposed a framework to detect the hole boundary points in the point cloud using a deep learning architecture to overcome the limitation of manually setting the threshold value after analyzing the nature of the point cloud. Generation of ground truth for real-time point clouds is a challenging task in data pre-processing step. We have trained the architecture to perform hole detection. The robustness of the model depends on the type of data used for training the network. We have filled the holes in the point cloud using the detected boundary points by our proposed method. As presented in this paper, the obtained results show that our framework works well even on the real point cloud data which is very different from the training data.

References

- [1] Setty, S., Ganihar, S.A., & Mudanagudi, U. (2015). Framework for 3D object hole filling. 2015 *Fifth National Conference on Computer Vision, Pattern Recognition, Image Processing and Graphics (NCVPRIPG)*, 1-4.
- [2] Yizhou Yu, A. Ferencz and J. Malik, (2001) "Extracting objects from range and radiance images," *IEEE Transactions on Visualization and Computer Graphics*, vol. 7, no. 4, pp. 351-364, Oct.-Dec.

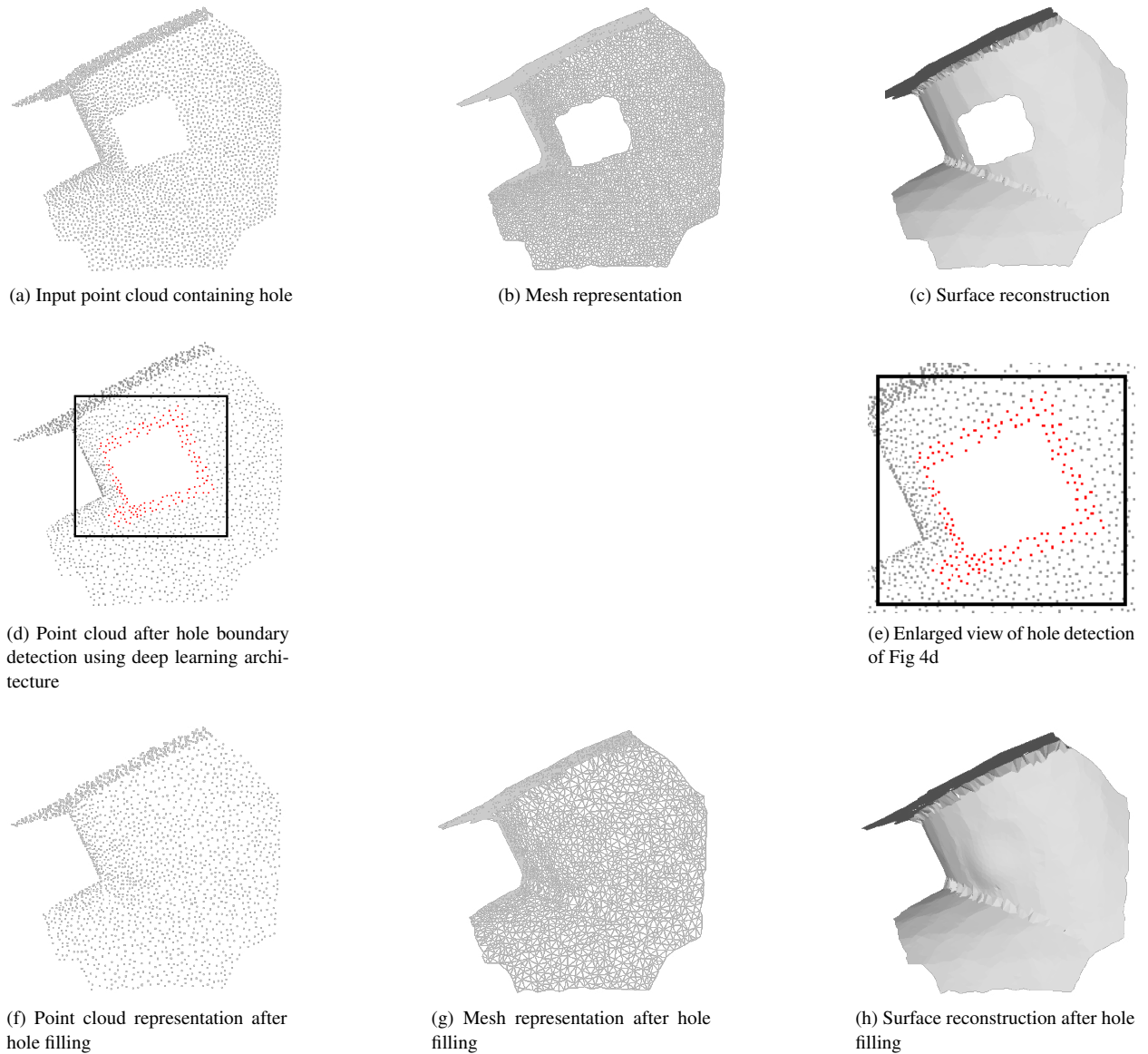


Fig. 4: Illustration of hole detection and filling in point cloud using our proposed framework

- [3] J. Branch, F. Prieto and P. Boulanger, (2006) "Automatic Hole-Filling of Triangular Meshes Using Local Radial Basis Function," *Third International Symposium on 3D Data Processing, Visualization, and Transmission (3DPVT'06)*, Chapel Hill, NC, pp. 727-734.
- [4] Emiliano Pérez, et al. (2016) "A comparison of hole-filling methods in 3D." *International Journal of Applied Mathematics and Computer Science* 26.4: 885-903.
- [5] SaitSuer, SinanKockara, MutluMete, (2011) "An improved boundary detection in dermoscopy images for density based clustering", *Proceedings of the Eighth Annual MCBIOS Conference. Computational Biology and Bioinformatics for a New Decade*, USA.
- [6] H.Xianfeng, C.Xiaoguang, Z.Fan, G.Jianya, (2008) "SideRatioConstrainBased Precise Boundary Tracing Algorithm for Discrete Point Clouds", *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, Volume. XXXVII, Part. B3b, Beijing.
- [7] A.Sampath, J.Shan, (2007) "Building Boundary Tracing and Regularization from Airborne Lidar Point Clouds", *Photogrammetric engineering and remote sensing*, Volume. 73, Number. 7, Pages. 805-812, USA.
- [8] SHEN Wei, (2008) "Building Boundary Extraction Based on LiDAR Point Clouds Data", *The International Archives of the Photogrammetry*,

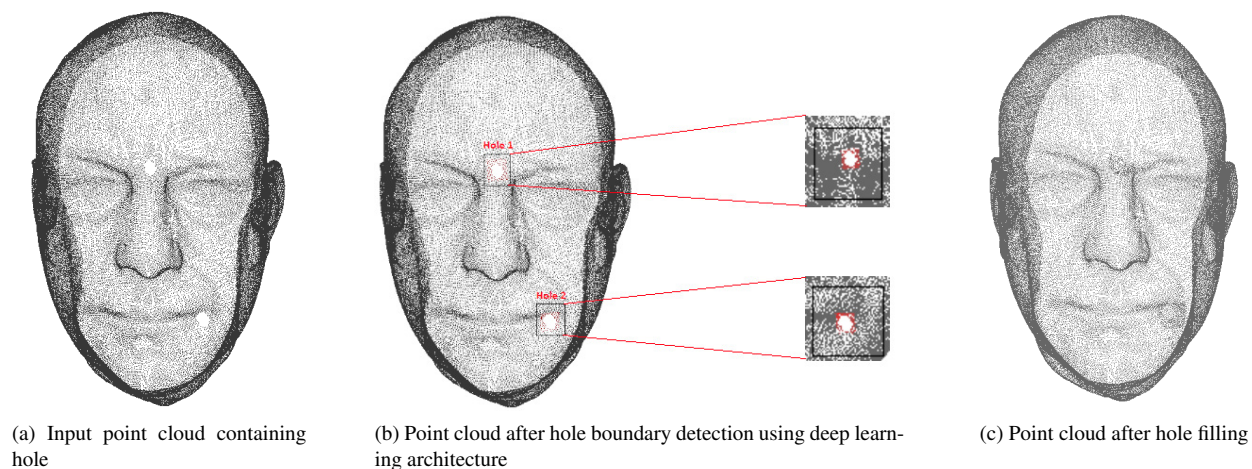


Fig. 5: Illustration of hole detection and filling in point cloud using our proposed framework on real data.

Remote Sensing and Spatial Information Sciences, Volume. XXXVII, Part. B3b, Beijing.

- [9] Evangelos Kalogerakis, Derek Nowrouzezahrai, Patricio Simari, Karan Singh, (2009) "Extracting lines of curvature from noisy point clouds", *Journal Computer-Aided Design archive*, Volume. 41, Issue. 4, USA, April.
- [10] Schnabel, Ruwen & Wahl, Roland & Wessel, Raoul & Klein, Reinhard. (2012). "Shape Recognition in 3D Point-Clouds".
- [11] H Bendels, Gerhard & Schnabel, Ruwen & Klein, Reinhard. (2006). "Detecting Holes in Point Set Surfaces". *Journal of WSCG*.
- [12] Aldeeb, Nader H. and Olaf Hellwich. (2017) "Detection and Classification of Holes in Point Clouds." *VISIGRAPP*.
- [13] Nguyen, Van-Sinh et al. (2012) "Boundary extraction and simplification of a surface defined by a sparse 3D volume." *SoICT*.
- [14] V. S. Nguyen, T. H. Trinh and M. H. Tran, (2015) "Hole Boundary Detection of a Surface of 3D Point Clouds," *International Conference on Advanced Computing and Applications (ACOMP)*, Ho Chi Minh City, 2015, pp. 124-129.
- [15] J. Branch, F. Prieto and P. Boulanger, (2006) "Automatic Hole-Filling of Triangular Meshes Using Local Radial Basis Function," *Third International Symposium on 3D Data Processing, Visualization, and Transmission (3DPVT'06)*, Chapel Hill, NC, pp. 727-734.
- [16] Davis, James & Marschner, S.R. & Garr, Matt & Levoy, Marc. (2002). "Filling Holes In Complex Surfaces Using Volumetric Diffusion". 428 - 441. 10.1109/TDPVT.2002.1024098.
- [17] Chalmoviansky, Pavel & Jüttler, Bert. (2003). "Filling Holes in Point Clouds". 196-212. 10.1007/978-3-540-39422-8-14.
- [18] M J, Adarsh & Kotturshettar, Sunidhi. (2017). "Automatic Hole Detection for Selective Hole Filling in Point Cloud Data - Results".
- [19] Yu, Lequan and Li, Xianzhi and Fu, Chi-Wing and Cohen-Or, Daniel and Heng, Pheng-Ann, (2018) "PU-Net: Point Cloud Upsampling Network", *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [20] Qi, Charles Ruizhongtai et al. (2017) "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation." *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*
- [21] J. Branch, F. Prieto and P. Boulanger, (2006) "Automatic Hole-Filling of Triangular Meshes Using Local Radial Basis Function," *Third International Symposium on 3D Data Processing, Visualization, and Transmission (3DPVT'06)*, Chapel Hill, NC, pp. 727-734.
- [22] Davis, James & Marschner, S.R. & Garr, Matt & Levoy, Marc. (2002). "Filling Holes In Complex Surfaces Using Volumetric Diffusion". 428 - 441. 10.1109/TDPVT.2002.1024098.
- [23] models, 3. (2019). 3D Face Scan — Professional 3D Scanning Solutions. [online] Artec3d.com. Available at: <https://www.artec3d.com/3d-models/face> [Accessed 14 Nov. 2019].